SC4052 Project 1, topic 8: X-as-a-Service

By Quah YiKai Gary, U2322511D

This project studies how to design and implement an "X-as-a-Service" (XaaS) solution, turning complex capabilities into scalable, on-demand cloud services.

Table Of Contents

1. Introduction **2**
Background 2
Objective 2
2. literature review **3**
2.1 Microsoft Teams 3
2.2 Zoom 4
4. Solution design **4**
4.1 Stack 4
4.1.1 Frontend 4
4.1.2 Backend 4
4.1.3 Database 5
4.2 Software architecture 5
5. Illustrative examples **6**
5.1 Basic meeting querying 6
5.2 Uploading new meeting notes to the database 7
5.3 Meeting is automatically added to the graph 8
5.4 Ability to immediately query new graphs 9
5.5 Ability to carry out specific queries for detailed information 10
6. Conclusions **10**
6.1 Challenges faced 10
6.1 Future outlook 11
7. References **11**

1. Introduction

Background

In my professional internship, my team and I found ourselves struggling with finding information from past meetings regarding clients and offshore team members. We had to go through an arduous process of emailing and messaging each individual participant in hopes they could point us in the right direction, often causing a delay in production due to differing time zones.

Although existing Software as a Service meeting applications provide meeting summaries and the ability to query individual meetings, such as Microsoft Teams and Zoom, they lack the ability to query and search through a collection of meeting summaries effectively.

Users would still have to dig through individual tens or hundreds of meetings to find the required information, causing frustration and delay in information retrieval.

Objective

This project aims to address the lack of a collective meeting notes query service, introducing a meeting notes query as a service. This service allows users to upload individual meeting summaries and query through them as a collective.

2. literature review

2.1 Microsoft Teams

Microsoft Teams has a recap tab available only to premium users, allowing users to view a recap of any meeting. Users will be able to see a Copilot-generated meeting summary generated from the transcript, along with the ability to interact with Copilot to query details about the meeting.

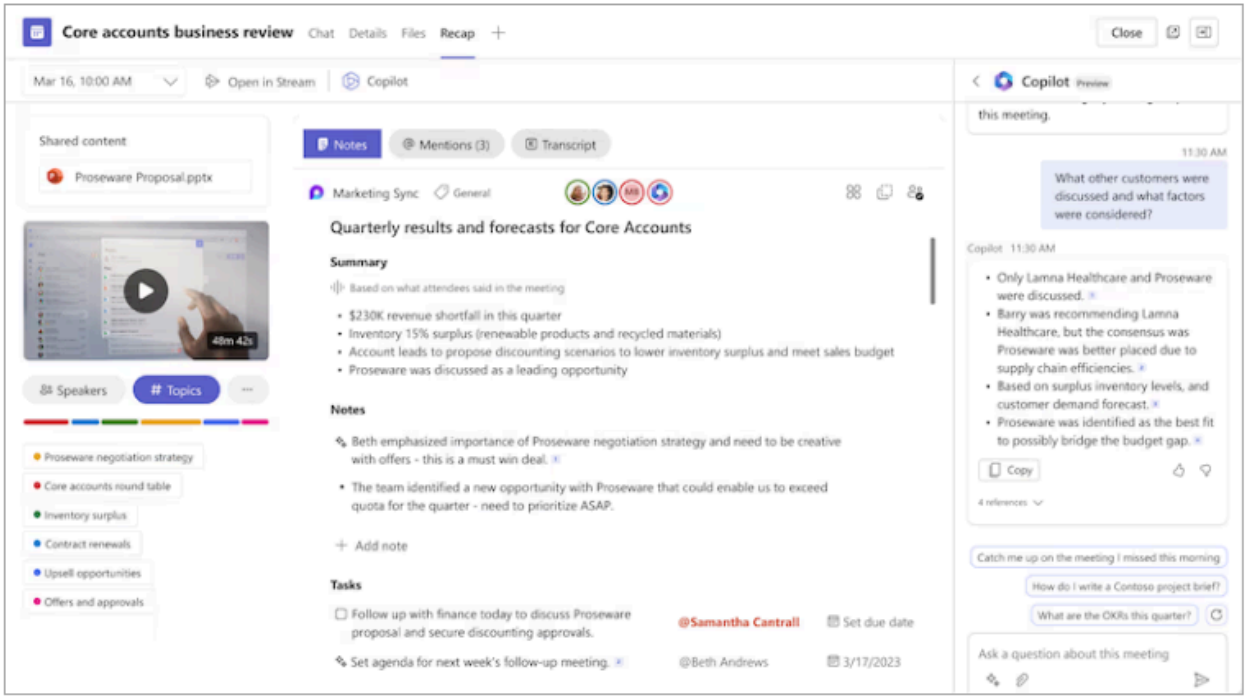


Figure 1: Copilot [1]

2.2 Zoom

Zoom provides a detailed meeting summary for premium users, along with the ability to view a compiled list of their meeting summaries. Users will be able to share, delete and modify these summaries. However, a feature regarding querying individual or compiled meeting summaries is not available.

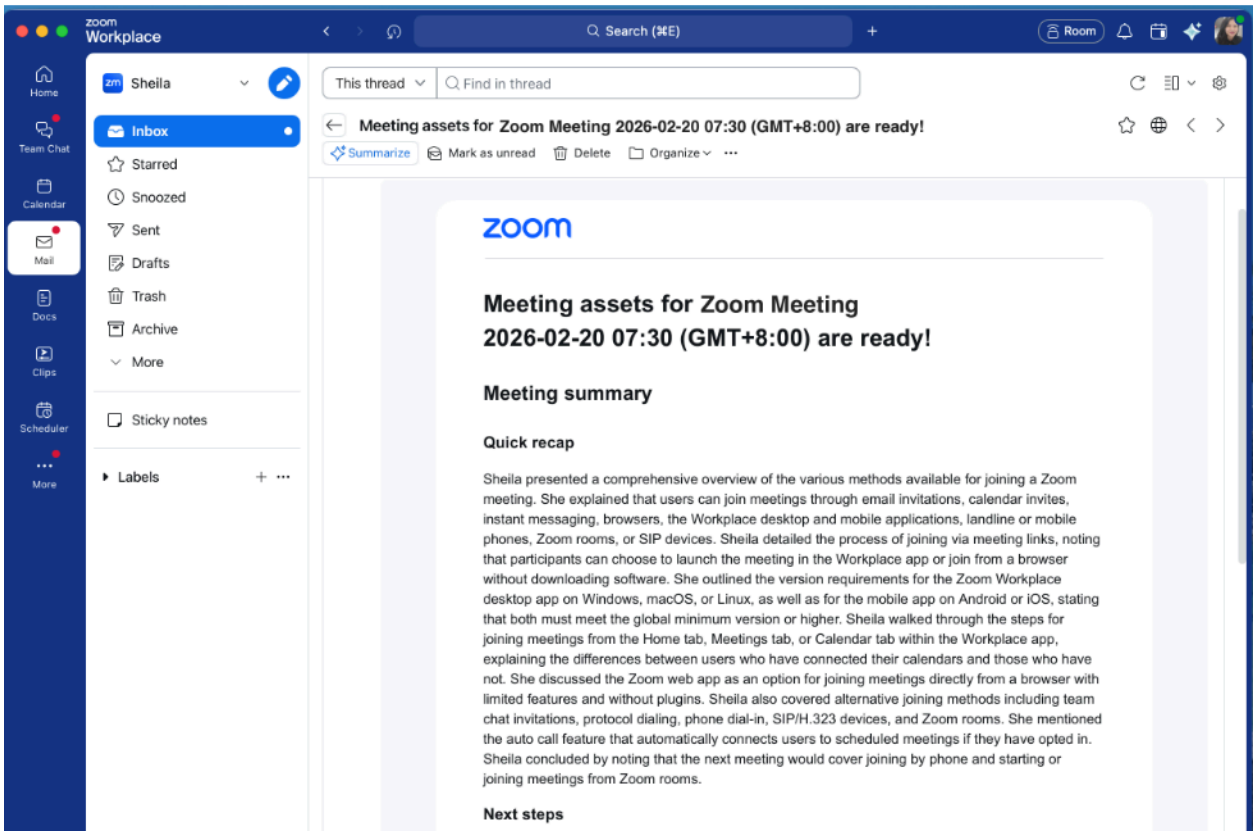


Figure 2: Zoom [2]

4. Solution design

4.1 Stack

4.1.1 Frontend

The frontend of the application makes use of REACT for a dynamic user interface and TypeScript for type safety.

4.1.2 Backend

The backend of the application makes use of a Python-based backend, consisting of FastAPI with Pydantic model validations for type checking and type safety.

4.1.3 Database

The software uses a local database, storing meetings in .txt files, allowing users who have no programming background to upload and view their meetings, whereas the Graph is stored in a cache.json file, making the graph structure easy to read and modify for programmers, reducing the need to regenerate the graph on every app reload.

4.2 Software architecture

Expanding on the course covered content, to implement a solution, we shall implement GraphRAG with pagerank as the source of truth for information retrieval, preventing hallucination during information retrieval by LLMs.

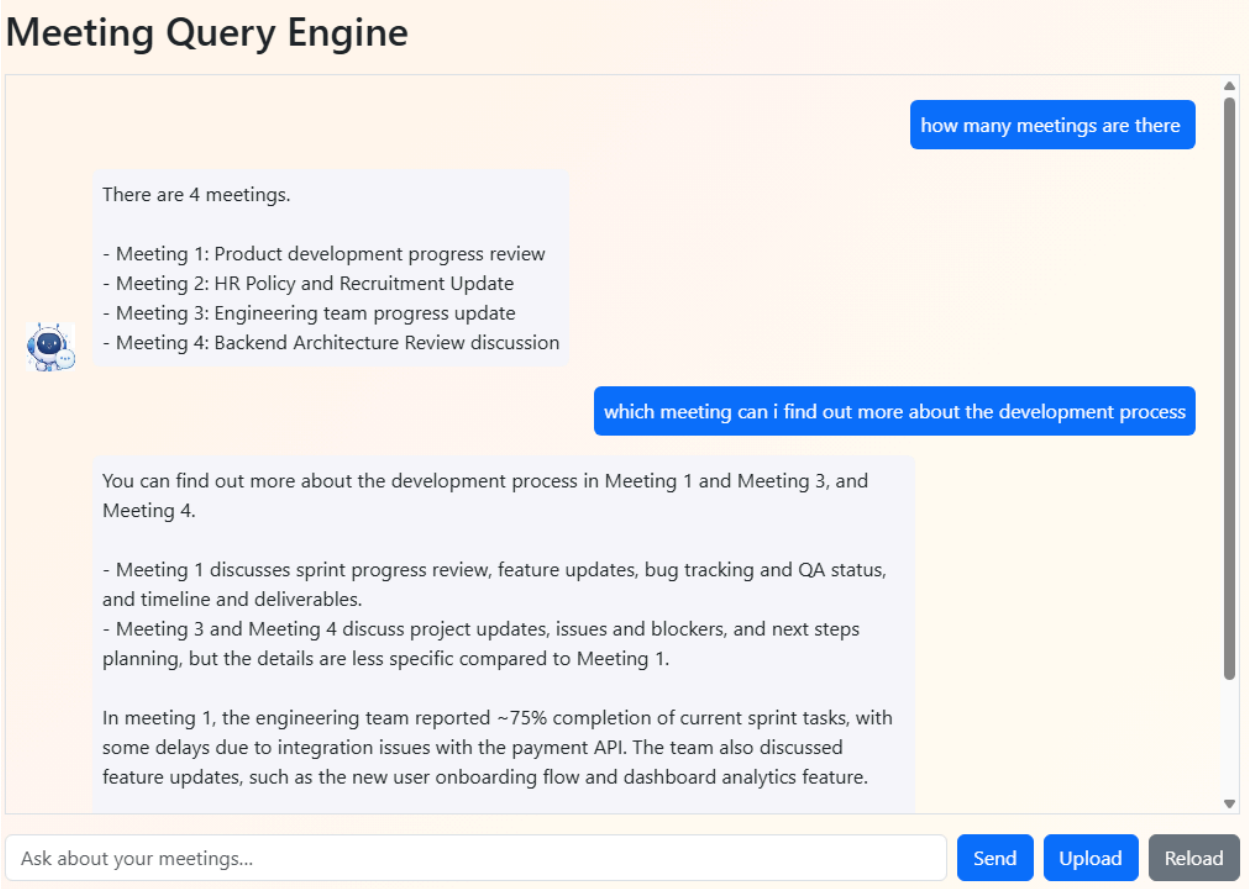
After a user uploads their meeting notes to the XaaS, an LLM will extract relevant objects, relations and subjects as triples from the uploaded meeting notes, storing them in a knowledge graph, where each meeting is a subgraph, combining multiple meetings into a single large graph.

When a user makes a query to the XaaS, the LLM will extract seed nodes from the user query and pass it to the backend, which runs the seed nodes against the GraphRAG to retrieve the nodes with the highest relevance through pagerank.

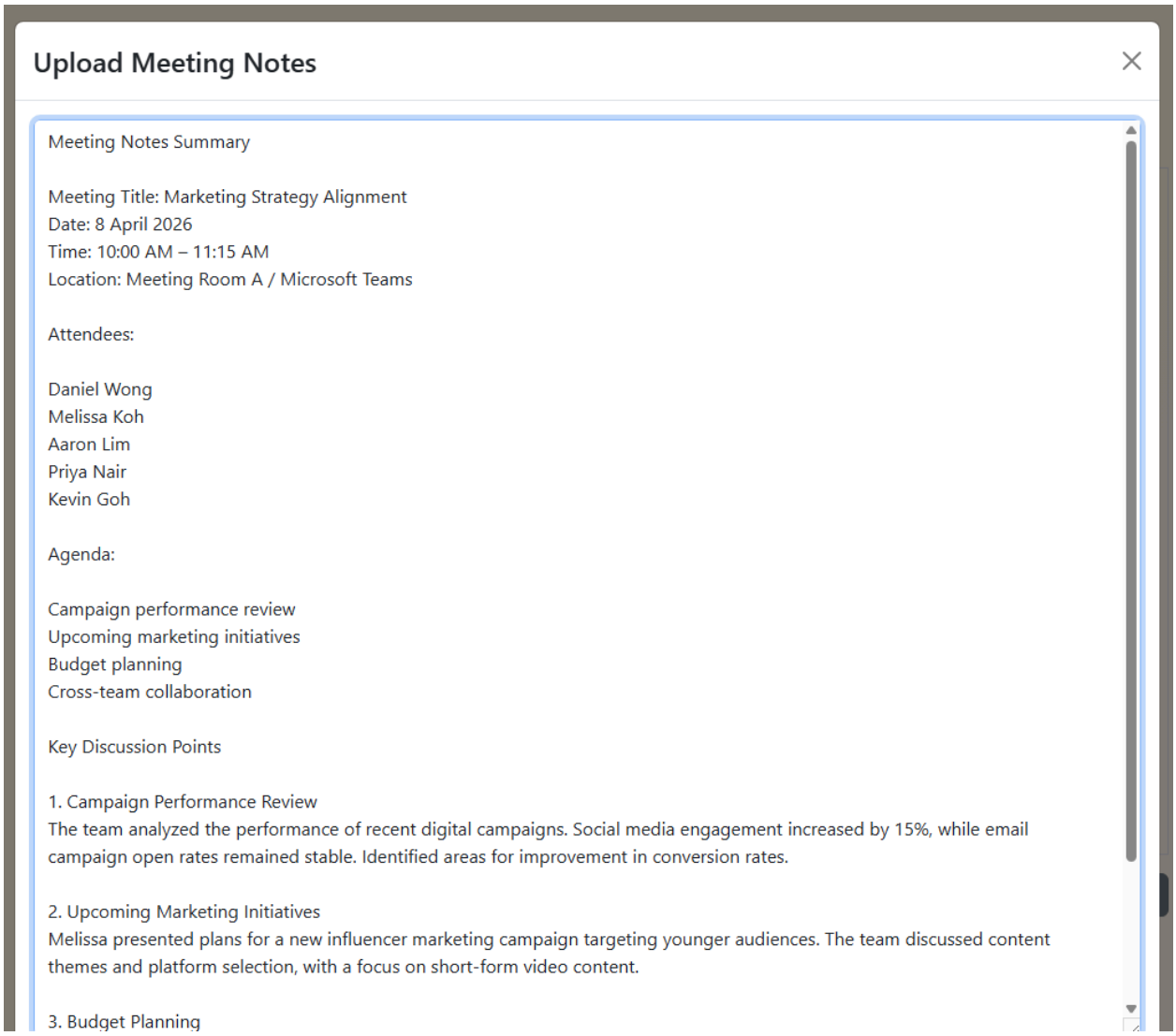
These nodes will be passed back to the LLM to return a natural language response to the user.

5. Illustrative examples

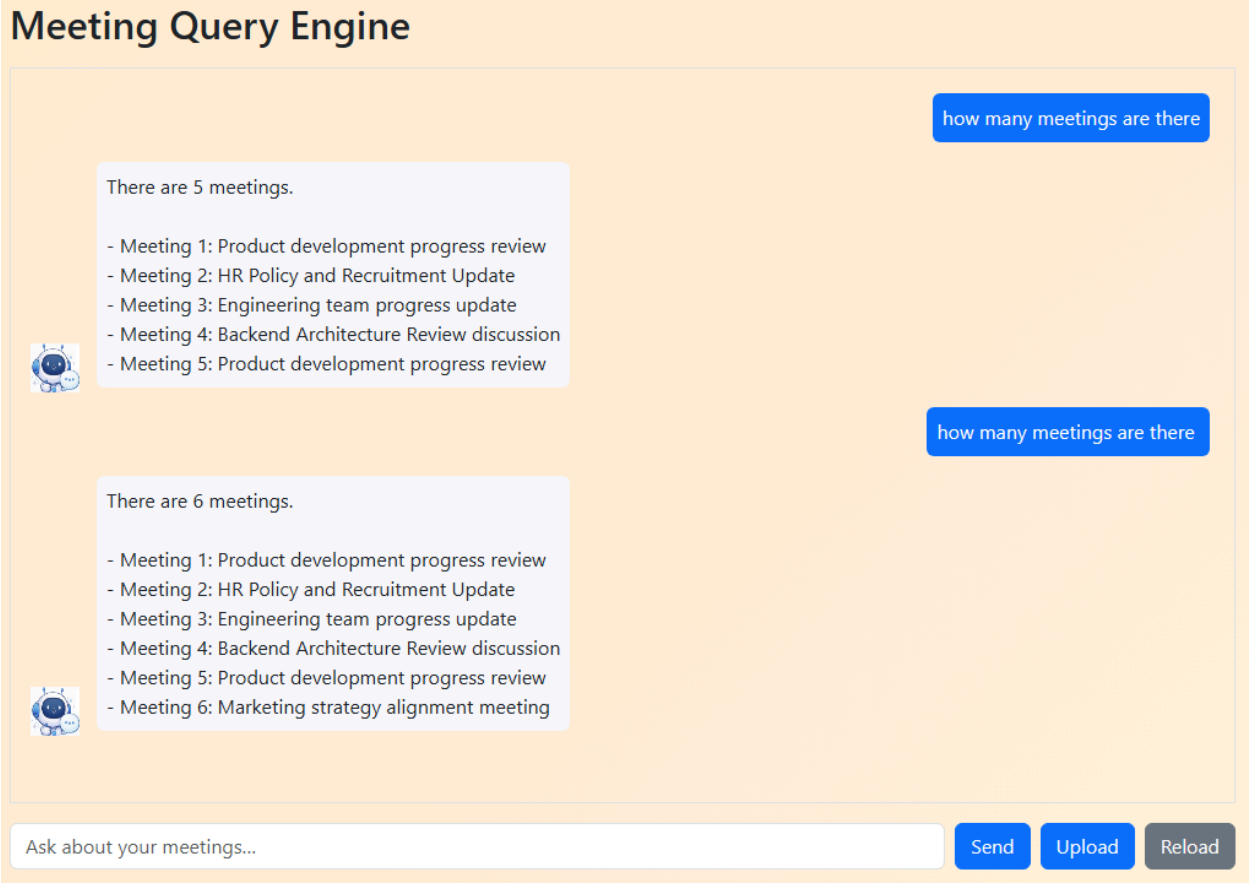
5.1 Basic meeting querying



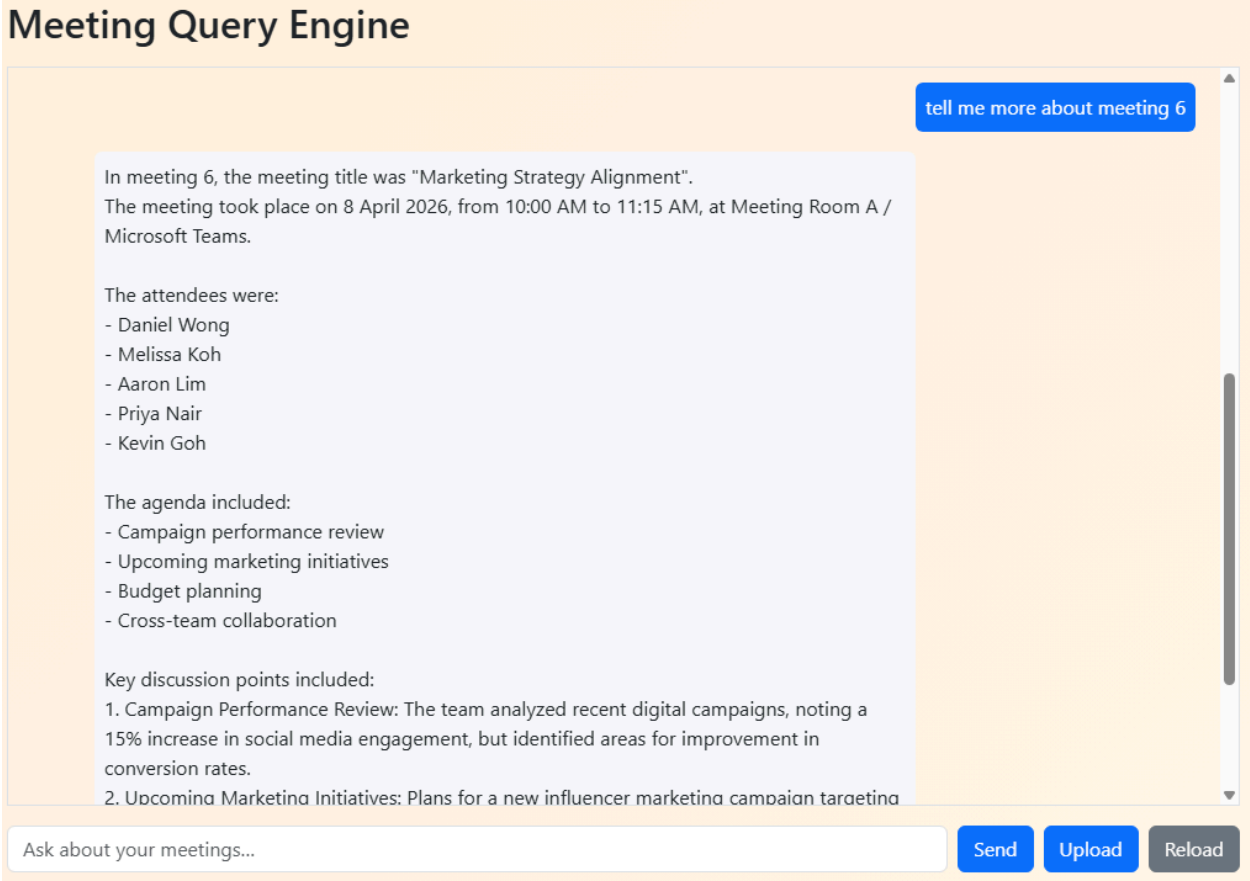
5.2 Uploading new meeting notes to the database



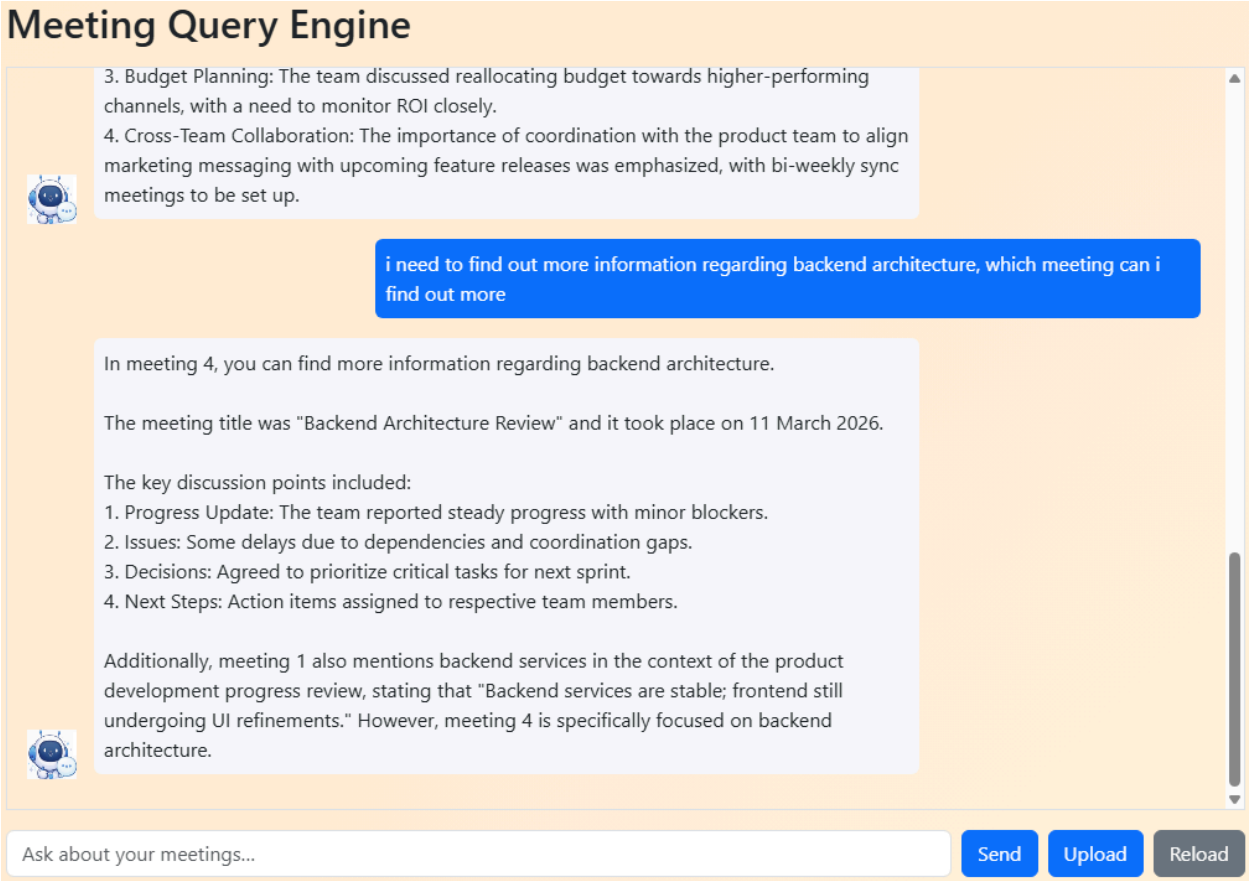
5.3 Meeting is automatically added to the graph



5.4 Ability to immediately query new graphs



5.5 Ability to carry out specific queries for detailed information



6. Conclusions

6.1 Challenges faced

Some challenges faced would be the prompt engineering for the LLM to perform as intended. Additional instructions would be required to be passed to the LLM, such as for formatting of output, rules and the LLM's role in the application.

```
SYSTEM_PROMPT = """You are a precise meeting-notes assistant. Answer questions strictly \
from the provided meeting data — never from general knowledge or inference.

Rules:
- Only use information present in the meeting index or notes provided.
- If the data doesn't contain enough information, say so plainly.
- Cite the meeting ID when stating a fact, e.g. "In meeting 2, ...".
- Never speculate or pad your answer.

Formatting:
- Use blank lines between sections.
- Use dashes for lists.
- No markdown (no **, ##, *)."""
```

Perhaps with a better model rather than a free version, it would be able to perform better or might even perform differently.

6.1 Future outlook

The application can be further improved by turning it into an application that retrieves meeting notes or summaries directly from meeting applications or turning it into a plugin for mainstream meeting applications.

Source code: https://github.com/GaryQuah/SC4052-Cloud-computing/tree/main/Project1

7. References

1. "Recap in Microsoft Teams - Microsoft Support," *Microsoft.com*, 2019. https://support.microsoft.com/en-us/office/recap-in-microsoft-teams-c2e3a0fe-504f-4b2c-bf85-504938f110ef
2. "Using Meeting Summary with AI Companion," *Zoom*, 2025. https://support.zoom.com/hc/en/article?id=zm_kb&sysparm_article=KB0058013